Language Models, Embeddings, and Representations on WikiText-2

DSA4213 Assignment 1 · [Your Name] · [Matric No.] · Code: [GitHub link / zip]

Guiding question. On a small corpus with a matched compute budget, how do model families and input-embedding choices differ in next-token prediction, and does better next-token prediction translate into better fixed-length representations for a downstream classifier?

1 Setup

Corpus and preprocessing. WikiText-2 (`wikitext-2-v1`, HuggingFace) [1]. The text is already tokenized and rare words are already mapped to `<unk>`. We lowercase everything (GloVe-6B is uncased, so this keeps the embedding-coverage comparison fair), split on whitespace, drop blank lines and append `<eos>` to every line. The vocabulary is built from the training split only: 28,913 types, including `<pad>`, `<unk>`, `<eos>`. The splits contain 2.08M / 216K / 244K tokens (train / valid / test), with `<unk>` rates of 2.6% / 5.4% / 6.2%.

Common evaluation setup. Each split is cut into windows of 65 tokens that overlap by one token, so every target in the stream is scored exactly once. The last window is right-padded, and `<pad>` targets are excluded from the loss. No hidden state is carried across windows. Every model, including the n-gram (whose history is clipped to the window), therefore predicts each target from at most the preceding 64 in-window tokens. We report mean next-token cross-entropy (CE, nats) and perplexity (PPL $= e^{\mathrm{CE}}$). Because no state is carried across windows, our PPLs are higher than published numbers for stateful LSTMs and should only be compared within this report.

Hardware and timing. Neural models were trained on an Apple M5 (16 GB) with the PyTorch 2.14 MPS backend; the n-gram and Word2Vec ran on the CPU. Training time is the wall-clock time of the training loop (validation excluded), with device synchronisation before and after timing. The common inference workload is forward-only over the first 50 test batches ($32 \times 64 = 102{,}400$ positions), after 5 warm-up batches. All runs use seed 42; generation uses seed $123 + i$ for prompt i.

2 Part I: Model families

Models. *3-gram*: interpolated Kneser–Ney [2, 3] with discount $D = 0.75$ (implemented from scratch). The highest order uses raw counts, lower orders use continuation counts, and the unigram level is interpolated with a uniform distribution. *Neural*: all three models use a randomly initialized, trainable 100-d embedding (the dimension is chosen to match GloVe-100d for Part II), followed by a backbone and an untied linear softmax layer over 28,913 words:

- *RNN*: 2-layer tanh RNN with 256 hidden units.
- *LSTM* [4]: 2-layer LSTM with 256 hidden units.
- *Transformer* [5]: linear projection from 100 to 256 dimensions, learned positional embeddings, then 2 pre-LN [6] decoder blocks (4 heads, FFN 512, GELU) with a causal mask.

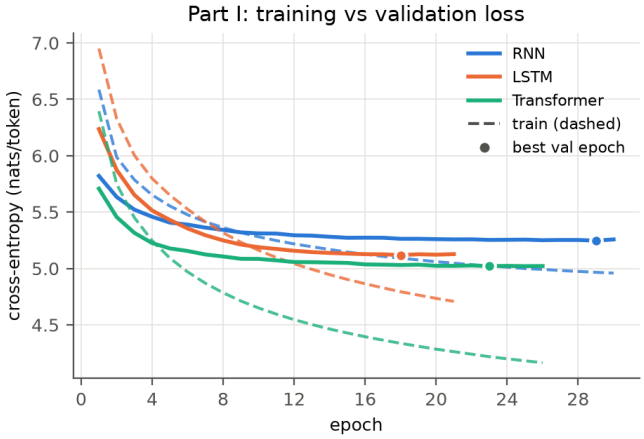
Dropout is 0.3 for the RNN and LSTM and 0.2 for the Transformer. All three are trained identically: Adam (lr 10^{-3}), batch size 32×64, gradient-norm clipping at 1.0, at most 30 epochs, and early stopping with patience 3 on validation CE. We keep the best-validation checkpoint and evaluate it once on the test set. We checked causality empirically: perturbing token t leaves all logits at positions $< t$ unchanged.

Table 1: Prediction quality and cost on the common setup. "Backbone" counts parameters excluding the shared-shape 100-d input embedding (2.89M) and the softmax layer (7.43M). The n-gram's inference figure is per-target scoring in Python on the CPU, so it is not directly comparable to the GPU forward passes.

Model	Test PPL	Test CE	Val PPL	Size	Backbone	Best / run ep.	Time to best	s/epoch	Infer. tok/s
3-gram KN	227.8	5.43	242.8	2.61M entries	–	–	2.3 s (counting)	–	861K (CPU)
RNN	176.5	5.17	190.1	10.55M	0.22M	29 / 30	3,548 s	122	89K
LSTM	156.7	5.05	167.1	11.21M	0.89M	18 / 21	2,191 s	121	77K
Transformer	**143.0**	**4.96**	**151.6**	11.42M	1.10M	23 / 26	2,633 s	115	91K

Observations.

- **Prediction quality.** The ranking is Transformer (143.0) < LSTM (156.7) < RNN (176.5) < 3-gram (227.8). Moving from the 3-gram to the RNN gives the largest single drop (−51 PPL).
- **Convergence.** The Transformer has the lowest validation loss at every epoch. It also overfits most: at its best epoch its validation CE is 0.80 nats above its training CE, versus 0.33 for the LSTM and 0.28 for the RNN, and its training loss keeps falling after validation flattens (Fig. 1). The LSTM starts worst (epoch-1 PPL 511 vs. 337 for the RNN) but overtakes the RNN at epoch 6. The RNN converges slowest (best epoch 29) and its validation curve is still noisy at the end.



Generated text (5 prompts × 40 tok)	3-gram	RNN	LSTM	Transf.
distinct-2	0.92	0.87	0.86	0.90
repeated-trigram fraction	0.02	0.04	0.06	0.04
<unk> fraction	0.07	0.07	0.09	0.02
heading "=" fraction	0.04	0.14	0.14	0.05

Figure 1: Left: training (dashed) and validation (solid) CE; the dot marks the best-validation epoch. Right: repetition statistics of continuations under matched decoding (top-k = 40 [12], temperature 0.8, 40 new tokens, same seeds; only <pad> is banned). Excerpts are in Table 2; full samples are in Appendix C.

Table 2: Continuations for the prompts "in the early years of the war" / "he was born in" (*italics*).

3-gram	`. in one season . for the first time . <eos> the family . in the early neolithic builders , he was " . <eos> = = <eos> = = …` / *`captivity as the film , the first ever " , as the " <unk> in the <unk> @-@ <unk> …`*
RNN	`. <eos> = = = microscopic life = = = <eos> on the other hand , <unk> can be seen in the top of the 20th century . the song was written by her parents …` / *`march 2006 . <eos> = = = new york = = = <eos> the australian football league games , federer began the second four @-@ yard @-@ winning @-@ touchdown …`*
LSTM	`, the german prerogative was not to be the son of " the <unk> " . the king was also influenced by the <unk> ( croatian @-@ <unk> ) on may 23 , 1863 …`
Transformer	`. <eos> = = = hurricane kenna = = = <eos> in the atlantic ocean , hurricane christine moved to the central pacific ocean . on january 16 , an eye of convection was detected …` / *`… the hurricane produced gusty winds of 140 mph ( 140 km / h ) …`*

- **Stability.** The RNN alone shows gradient spikes: its pre-clip gradient norm reaches 212.7, against at most 1.7 for the other models.
- **Cost.** Wall time per epoch is similar for all three neural models (115–122 s). Inference throughput (77–91K tok/s) varied by up to 20% between two identical runs (Transformer: 74K vs. 91K tok/s), so we do not rank the neural models by speed. The 3-gram is about 1,000× cheaper to "train" and about 10× faster to score, but it stores 2.6M entries.
- **Generated text** (Table 2).
  - The 3-gram produces locally plausible trigrams with no phrase-level coherence ("he was born in captivity as the film").
  - The RNN and LSTM produce fluent spans of 5–10 tokens but drift topic within a sentence (the RNN moves from "australian football league" to "federer" to "touchdown"). About 14% of their tokens are section-heading markup.
  - The Transformer stays on one topic for an entire continuation (hurricane, chart positions, genus/species) and emits the fewest <unk> tokens (1.5%). It still makes local repetitions ("a range of a range of", "corpus christi and corpus christi") and inconsistent facts ("140 mph (140 km / h)", "debuted at number 1 … peaked at number 21").

  The repetition statistics differ only slightly between models and come from just 200 tokens per model, so we treat them as illustrative.

Possible explanations (not tested separately).

- **Transformer advantage.** Self-attention gives every position direct access to all 64 preceding tokens, which fits the observed topical consistency. However, the Transformer also has the largest backbone (1.10M vs. 0.89M for the LSTM and 0.22M for the RNN), so architecture and parameter count are confounded.
- **Transformer overfitting.** Its larger train–validation gap suggests higher memorisation capacity on a 2M-token corpus. Every model saw at most about 62M training tokens (30 epochs), well below the roughly 20 tokens per parameter suggested as compute-optimal for 11M-parameter models [11], so this is a data-limited regime.
- **RNN instability.** The RNN's spikes match the exploding-gradient behaviour of tanh recurrences [7]. Clipping was essential to train it at all, and the instability probably contributes to its slow, noisy convergence.
- **Similar epoch times.** About 70% of each neural model's parameters are in the 28.9K-way softmax layer, which plausibly dominates compute at this scale. The Transformer's parallelism therefore does not show up as faster training at $T = 64$ on one GPU.

Trade-off. On this budget, the Transformer buys about 9% lower PPL than the LSTM and clearly better topical consistency at similar wall time. It needs early stopping because it overfits fastest. The n-gram is the only choice when training compute is nearly zero, but it degrades sharply in quality.

3 Part II: Embedding variants (LSTM)

Design. We use the Part I LSTM and change only the input-embedding table (100-d). Architecture, data, optimiser, seed (the model is re-seeded right before construction), stopping rule and the 30-epoch cap are identical across the three settings:

- **Random, trainable.** We reuse the Part I LSTM.
- **Own Skip-gram, frozen** [8]. gensim Skip-gram trained on the WikiText-2 training split only: window 5, 10 negative samples, 10 epochs, min_count 1, one thread. It covers 100% of the vocabulary.
- **GloVe, frozen** [9]. GloVe 6B 100-d (Wikipedia + Gigaword, 6B tokens). It covers 96.2% of vocabulary types and 94.7% of training tokens. The 1,103 out-of-vocabulary (OOV) types are dominated by very frequent WikiText markup: `<unk>` (54.6K occurrences), `<eos>` (23.8K), `@-@` (16.9K), `@.@`, `@,@` and `'t`.

OOV rows are drawn from $\mathcal{N}(0, \sigma)$, where σ is the standard deviation of the pretrained vectors (0.41 for GloVe), and the `<pad>` row is zero. In both frozen settings the *entire* table is fixed, including the random OOV rows, which leaves 8.32M trainable parameters instead of 11.21M.

Table 3: Embedding ablation on the LSTM. The cost of training our own embeddings is reported separately from LM training. Gradient norms are measured before clipping and averaged per epoch.

Input embedding	Test PPL	Val PPL	Val PPL ep1/ep5	Best/run	Val−train CE	Grad norm mean/max	Trainable	LM time to best	Embedding cost	distinct-2
Random, trainable	156.7	167.1	511 / 228	18 / 21	0.33	0.25 / 1.26	11.21M	2,191 s	–	0.86
Own Skip-gram, frozen	**141.1**	**149.7**	580 / 200	18 / 21	0.25	0.25 / 1.30	8.32M	1,876 s	55 s (1 CPU thread)	0.89
GloVe 6B, frozen	153.1	162.5	653 / 223	18 / 21	0.20	0.25 / 1.62	8.32M	1,897 s	external (6B tokens)	0.75

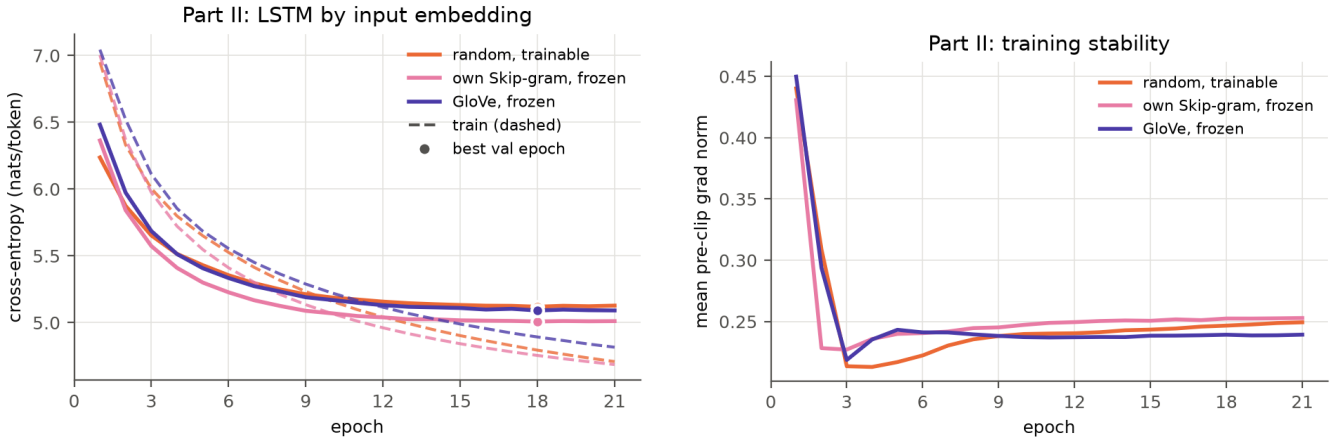


Figure 2: Part II training dynamics. Left: training (dashed) and validation (solid) CE. Right: mean pre-clip gradient norm per epoch.

Observations.

- **Prediction quality.** Both frozen settings beat the trainable baseline: Skip-gram reduces test PPL by 15.6 (−10%) and GloVe by 3.6. The Skip-gram LSTM (141.1) even edges past the Part I Transformer (143.0).
- **Convergence.** The frozen settings start worse (epoch-1 PPL 580 and 653 vs. 511), but Skip-gram overtakes the baseline at epoch 2. All three settings reach their best epoch at 18, and freezing narrows the generalisation gap (0.33 → 0.25/0.20 nats).
- **Stability.** Gradient behaviour is nearly identical across settings (mean norm 0.25, maximum 1.26–1.62; Fig. 2). Embedding choice does not affect stability here, unlike the architecture choice in Part I.
- **Cost.** Training our own embeddings costs 55 s, about 3% of the LM's time to best.
- **Generated text.** The GloVe model's samples are the most repetitive (distinct-2 0.75), but 24% of its sampled tokens are heading markup ("= = = synopsis = = =" repeated). With only 5 samples, we do not draw conclusions from this.

Interpretation: trainability vs. extra data.

- **Why freezing helps.** Freezing removes 2.89M parameters (26%) that would otherwise be fitted to only 2M tokens. The smaller generalisation gaps are consistent with reduced overfitting, and the Skip-gram result shows that a good fixed initialisation is worth more here than trainability. A plausible mechanism is that Skip-gram gives every word, including rare ones (min_count = 1, 10 passes), a vector shaped by its contexts. In the trainable table, rare words receive only a few noisy updates from the LM loss.
- **Why GloVe helps less despite its data.** GloVe was trained on about 3,000× more text, yet it helps less than Skip-gram on WikiText-2 PPL. Two plausible reasons:
  1. The OOV rows are precisely the most frequent structural tokens, so 5.3% of input tokens enter the LSTM as fixed random vectors that can never be refined.
  2. Skip-gram was trained on exactly the LM's token stream and its tokenization conventions, whereas GloVe reflects a different corpus mixture.

  The extra data in GloVe therefore does not translate into in-domain prediction, but it does matter for out-of-domain features (Part III).
- **What this design cannot separate.** It cannot fully disentangle "fixed vs. trainable" from "initialisation quality". A pretrained-but-trainable variant would separate the two; we did not run it (Section 5).

4 Part III: Downstream representation analysis

Task and protocol. AG News topic classification [10] (4 balanced classes) with stratified subsets: 10K train and a disjoint 2K validation set from the official training split, and 2K test from the official test split.

- **Tokenization.** Texts are tokenized to mimic WikiText conventions (punctuation and clitics split, `@-@` / `@,@` markers, HTML entities removed), mapped into the LM vocabulary and truncated to 64 tokens. Texts average 44.6 tokens; 6.2% are truncated and 8.4% of tokens are `<unk>` (vs. 6.2% on WikiText-2 test).
- **Representations.** Every LM stays frozen. A text becomes one fixed-length vector in one of three ways:
  - **last**: the top-layer hidden state at the last non-pad position.
  - **mean**: the average of top-layer states over non-pad positions, masked so padding is excluded.
  - **emb-mean**: the average of the LSTM's *input* embeddings, used as a context-free reference.

  Right padding never influences these states, because every model is causal.
- **Classifier.** All representations share the same classifier family and procedure: standardisation followed by multinomial logistic regression, with $C \in \{0.01, 0.1, 1, 10\}$ selected on validation macro-F1. The test set is scored once. We report 95% bootstrap confidence intervals (1,000 resamples of the test set) and paired bootstrap tests for the key differences.

Table 4: Frozen-LM features with a logistic-regression probe on AG News.

Representation (frozen LM, pooling)	Dim	Test acc.	Test macro-F1 [95% CI]	F1 Business / Sports	LM test PPL
LSTM, last	256	0.649	0.644 [0.623, 0.663]	0.48 / 0.79	156.7
LSTM, mean	256	0.705	0.703 [0.686, 0.723]	0.60 / 0.83	156.7
Transformer, last	256	0.720	0.718 [0.700, 0.735]	0.60 / 0.85	143.0
Transformer, mean	256	0.774	0.773 [0.755, 0.790]	0.69 / 0.88	143.0
RNN, mean	256	0.730	0.727 [0.708, 0.746]	0.61 / 0.85	176.5
LSTM + own Skip-gram (frozen), mean	256	0.791	0.790 [0.772, 0.806]	0.69 / 0.90	141.1
LSTM + GloVe (frozen), mean	256	**0.831**	**0.830 [0.812, 0.847]**	0.75 / 0.93	153.1
LSTM input embeddings, emb-mean (no context)	100	0.674	0.672 [0.651, 0.692]	0.57 / 0.76	–

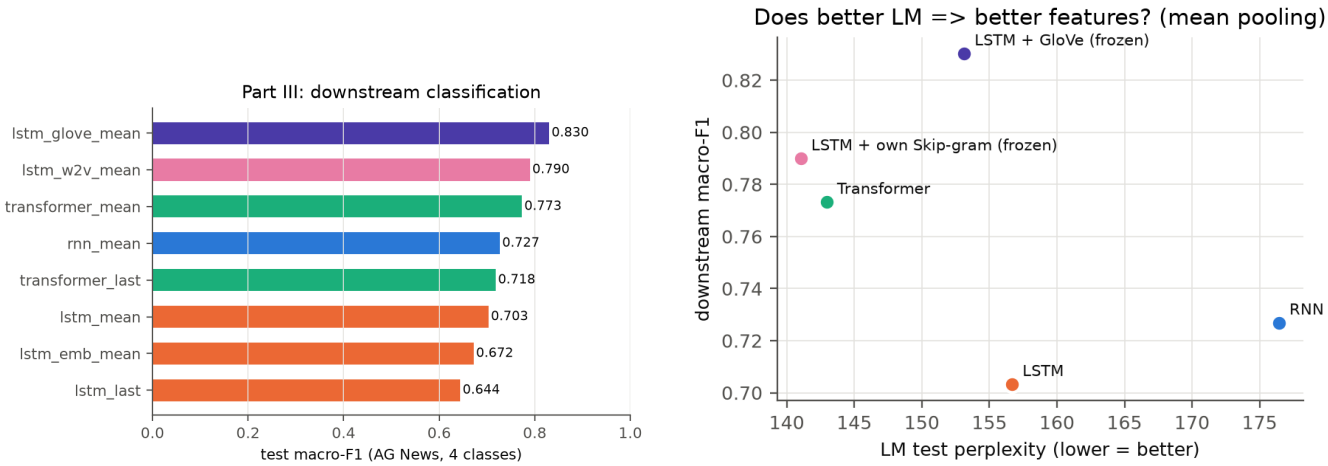


Figure 3: Left: test macro-F1 of each representation. Right: LM test PPL against downstream macro-F1, one point per LM (mean pooling).

Observations.

- **Primary comparison: architecture and pooling.**
  - The Transformer beats the LSTM under both poolings: +7.0 macro-F1 points with mean pooling (paired 95% CI [+4.8, +8.9]) and +7.4 with last pooling ([+5.0, +9.6]).
  - Mean pooling beats last-state pooling for both models: +5.9 for the LSTM ([+3.9, +8.0]) and +5.5 for the Transformer ([+3.7, +7.3]).
  - Context helps: LSTM mean states outperform the average of the same model's input embeddings by +3.1 ([+1.0, +5.3]).
- **LM ranking vs. downstream ranking.** The two rankings agree only partially. For the five LMs with mean pooling, Spearman $\rho = 0.6$ between the PPL rank and the F1 rank ($n = 5$, $p = 0.28$; Fig. 3, right).
  - The Transformer is better than the LSTM on both measures.
  - The RNN has the *worst* PPL but beats the LSTM downstream (+2.4, [+0.5, +4.2]).
  - The largest violation: GloVe ranks only 3rd in PPL but 1st downstream, ahead of Skip-gram (+4.0, [+2.4, +5.6]), which has the best PPL.
  - Skip-gram vs. Transformer (+1.7) is not distinguishable ([−0.2, +3.5]).
- **Errors.** Business is the hardest class for every representation (F1 0.48–0.75). Its most frequent confusion is with Sci/Tech: 84–117 of the 500 Business test items are predicted as Sci/Tech.
  - 121 test items (6%) are misclassified by all seven contextual representations. Many of these sit on genuine topic boundaries. "IBM posts broad Q3 revenue growth" is gold Sci/Tech but predicted Business. "DaimlerChrysler, GM to team up on hybrid engines" is gold Business but predicted Sci/Tech.

- Some look like label noise: "Stocks Close Higher on Brokerage Earnings" and "Tokyo Stocks Finish 2.6 Percent Higher" are gold World.
- Decisive entity names are often `<unk>` in the WikiText vocabulary (e.g. "Yuganskneftegaz", "Hynix"), which removes the strongest cue.
- Across representations, GloVe-mean is right where LSTM-last is wrong on 461 items, against 96 in the reverse direction. Typical rescues are sports headlines with misleading surface words, such as "Red Sox Stumble and Fumble Their Way to Series Lead" (LSTM-last: Sci/Tech).

Interpretation.

- **Why mean pooling wins.** The last state is trained to predict the *next* token, so it mostly encodes local syntax near the end of the snippet, often just after a full stop. Averaging aggregates the topical words spread across the text.
- **Why the rankings disagree.** Lower WikiText-2 PPL is neither necessary nor sufficient for better topic features. Downstream accuracy depends heavily on lexical knowledge of news vocabulary and entities. GloVe acquired this from 6B tokens that include newswire (Gigaword), the same genre as AG News, while PPL measures only in-domain Wikipedia prediction. In-domain Skip-gram helps both measures. The RNN result shows that ranking reversals happen even without external data, which fits a broader point: next-token loss rewards local predictive detail, which is not the same as text-level topic information. We cannot isolate a mechanism for the RNN case from these runs.

5 Limitations, failure cases, and lessons

- **Single seed and limited timing.** Each configuration was trained once (seed 42), and the MPS kernels are not bit-exact, so small PPL differences (e.g. Skip-gram LSTM vs. Transformer, 141.1 vs. 143.0) are not established. Throughput was timed once per model and varied by up to 20% between identical runs.
- **No per-architecture tuning.** All models share one optimiser setting with a constant learning rate and no schedule. Results describe this matched budget and do not show each architecture's best achievable performance.
- **Choices that limit comparability.** Lowercasing, a word-level vocabulary with `<unk>`, and 64-token stateless windows make our PPLs incomparable to the literature. Word-level `<unk>` also hurts transfer to AG News (8.4% `<unk>`).
- **Part II gaps.** We did not run trainable pretrained embeddings, so "frozen" and "good initialisation" are confounded. GloVe's OOV tokens are concentrated on frequent markup, which disadvantages it for this corpus specifically.
- **Part III gaps.** A linear probe on frozen features measures linearly accessible information only, and evaluation uses a 2K-item test subset (95% CIs of about ±1.8 points). Some "errors" are label ambiguity.
- **Lessons.**
  1. With 2M tokens, regularisation by freezing a good in-domain embedding beat adding trainable capacity.
  2. Gradient clipping was not optional for the vanilla RNN.
  3. Representation quality for a downstream task depends on pooling (±6 points) and on the lexical coverage of pretraining data at least as much as on next-token PPL.

References

[1] S. Merity, C. Xiong, J. Bradbury, R. Socher. Pointer Sentinel Mixture Models. arXiv:1609.07843, 2016.

[2] R. Kneser, H. Ney. Improved backing-off for M-gram language modeling. ICASSP, 1995.

[3] S. F. Chen, J. Goodman. An empirical study of smoothing techniques for language modeling. Computer Speech & Language, 1999.

[4] S. Hochreiter, J. Schmidhuber. Long short-term memory. Neural Computation, 1997.

[5] A. Vaswani et al. Attention is all you need. NeurIPS, 2017.

[6] R. Xiong et al. On layer normalization in the Transformer architecture. ICML, 2020.

[7] R. Pascanu, T. Mikolov, Y. Bengio. On the difficulty of training recurrent neural networks. ICML, 2013.

[8] T. Mikolov et al. Distributed representations of words and phrases and their compositionality. NeurIPS, 2013.

[9] J. Pennington, R. Socher, C. D. Manning. GloVe: Global vectors for word representation. EMNLP, 2014.

[10] X. Zhang, J. Zhao, Y. LeCun. Character-level convolutional networks for text classification. NeurIPS, 2015.

[11] J. Hoffmann et al. Training compute-optimal large language models. NeurIPS, 2022.

[12] A. Fan, M. Lewis, Y. Dauphin. Hierarchical neural story generation. ACL, 2018.

[13] Software: PyTorch; gensim (R. Řehůřek, P. Sojka, 2010); scikit-learn; HuggingFace `datasets`; matplotlib.

Appendix A Configuration and reproducibility

Setting	Value
Data	WikiText-2 v1 (lowercased); vocab 28,913 from train; windows 64+1, stride 64, no state carry-over
Shared LM settings	emb 100, untied output, Adam lr 10^{-3}, wd 0, batch 32, clip 1.0, ≤30 epochs, early stop patience 3 (val CE), seed 42
RNN / LSTM	2 layers, hidden 256, dropout 0.3 (embedding, between layers, output)
Transformer	100→256 projection, learned positions (64), 2 pre-LN blocks, 4 heads, FFN 512, GELU, dropout 0.2, final LayerNorm
3-gram	Interpolated Kneser–Ney, $D = 0.75$; uniform interpolation at unigram level
Skip-gram	gensim 4.4, sg=1, dim 100, window 5, negative 10, 10 epochs, min_count 1, 1 worker, `PYTHONHASHSEED=0`
Generation	5 prompts (`configs/prompts.txt`), top-k 40, T 0.8, 40 tokens, seed 123 + i, context clipped to 64
Part III	AG News 10K/2K/2K stratified (seed 42), max 64 tokens, StandardScaler + LogisticRegression (lbfgs, max_iter 5000), C by val macro-F1

Everything is reproduced with `bash scripts/run_all.sh` (about 4.5 h on the hardware above). All hyperparameters are in `configs/`, and every metrics JSON stores its resolved config and hardware info. See `README.md` for data access and commands.

Appendix B Effect of the training budget (15 vs. 30 epochs)

An earlier run capped at 15 epochs stopped every neural model at epoch 15, so none had converged. We therefore raised the cap to 30 epochs. The ranking is unchanged, and the gains are 1–3%: The Part III ordering was also stable. With the 15-epoch models the macro-F1 values were

Test PPL	RNN	LSTM	Transformer	LSTM + Skip-gram	LSTM + GloVe
15-epoch cap	180.0	158.0	145.7	142.1	155.3
30-epoch cap (reported)	176.5	156.7	143.0	141.1	153.1

GloVe 0.834, Skip-gram 0.789, Transformer-mean 0.760, RNN-mean 0.693, LSTM-mean 0.689 and LSTM-last 0.608. The same Transformer throughput measured 73.9K tok/s in that run, compared with 91.3K here.

Appendix C Generated samples (matched decoding)

Model	Prompt → continuation (first 25 tokens)
3-gram	`[the history of the city] of sarnia , which was then re @-@ released as an official single " no one wanted to do so , the <unk> and …`
RNN	`[the album was released] by paul mccartney . she was nominated for the uk of the music and <unk> . <eos> = = = = <unk> <unk> = = = …`
LSTM	`[the species is found in] the genus . <eos> = = = chemical bodies = = = <eos> although a protein of cadmium is a dag , the cell is …`
Transformer	`[the album was released] on august 17 , 2006 . " crazy in love " debuted at number 1 on the billboard hot 100 , 2009 . it peaked at number 21 …`
Transformer	`[the species is found in] the united states . the formation of species is a member of the migratory species of species that is particularly in the genus stereum …`
LSTM + Skip-gram	`[the album was released] on cd @-@ rom on july 7 , 2013 , and the video has been released on june 14 , 2003 , which the song was released …`
LSTM + GloVe	`[the album was released] by michael kahne . <eos> = = = synopsis = = = <eos> = = = production = = = <eos> = = = critical reception = = = …`

All samples are in `results/samples/*.txt`, and the full Part III error lists are in `results/samples/part3_errors.md`.

Appendix D Responsible tool and data use

All datasets are public and are downloaded by the code: WikiText-2 (CC BY-SA), AG News (non-commercial research use) and GloVe (PDDL). None are redistributed in the repository. No API keys or private data are used. **AI assistance:** the code base (data pipeline, models, training and evaluation scripts) and a first draft of this report were produced with the help of Claude (Anthropic), an AI coding assistant, under my direction. I chose the experimental design, reviewed and ran all experiments, verified the reported numbers against the generated result files, and revised the analysis to reflect my own understanding. [Edit this statement so that it accurately describes your own contribution.]